

PROGRAMAÇÃO MODULAR

UMA INTRODUÇÃO:
POO, CLASSES E OBJETOS.
MODULARIDADE.
QUALIDADE DE SOFTWARE.

PROF. JOÃO CARAM

CONHECENDO VOCÊS

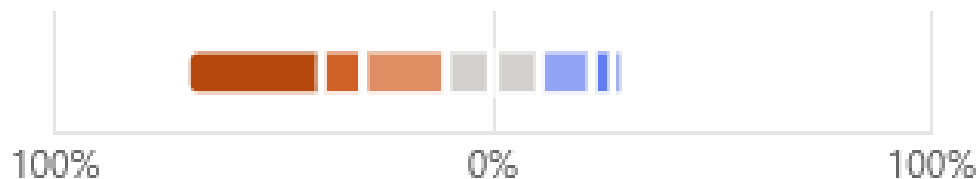
2

1. Numa escala de 0 a 10, como você avalia a você mesmo em programação e algoritmos?

[Mais detalhes](#)

● 0 - 4 ● 5 ● 6 ● 7 ● 8 ● 9 ● 10

Minha habilidade



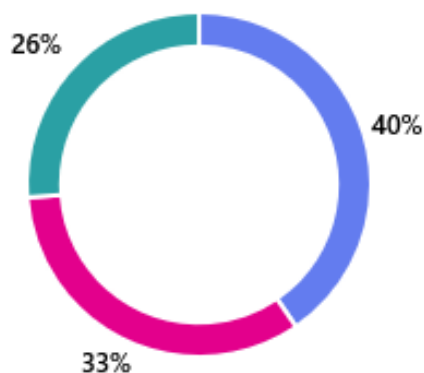
CONHECENDO VOCÊS

3

3. Você conhece alguma coisa sobre Programação Orientada Por Objetos (POO)?

[Mais
detalhes](#)

● Sim, sou familiar com alguns conceitos	17
● Já ouvi este conceito, mas não usei a fundo	14
● Nunca ouvi falar	11



CONHECENDO VOCÊS

4



4. O que você conhece dos seguintes conceitos de POO?

● Não conheço ● Conheço, mas não sei usar direito ● Conheço e sei usar bem

Classes e objetos

Construtores



PROGRAMAÇÃO ORIENTADA POR OBJETOS

5

- ▣ Para você, o que é uma classe?
- ▣ O que é um objeto?
- ▣ Para que serve criar um software baseado em classes e objetos?
 - ▣ Ou seja, usando programação orientada por objetos?

CLASSE

6

- ▣ Descrição de um Tipo Abstrato de Dados (TAD).
- ▣ Constituída por atributos (dados/características) e métodos (ações/comportamento).

CLASSE

7

- Um conjunto de entidades semelhantes compõem uma classe de objetos.
- Ex: Classe imóvel:
 - Ações: calcular IPTU, calcular valor por m², calcular idade, estimar aluguel.
 - Características: n^o de registro, endereço, área construída, quantidade de cômodos, valor de venda, ano de construção;

OBJETOS

8

- Um objeto representa uma entidade referenciável (identificada) de uma classe.
- Instância de uma classe.

CAM013

Av. Olegário Maciel

2.925m²

30

2.100.000.000

1908

PUCLU26

R. Cláudio Manoel

10.635m²

58

15.000.000

2016

OBJETOS

9

CAM013

Av. Olegário Maciel

2.925m²

30

2.100.000.000

1908

Idade: 117 anos

PUCLU26

R. Cláudio Manoel

10.635m²

58

15.000.000

2016

Idade: 10 anos

MODULARIDADE

10

“Mecanismo para aumentar a flexibilidade e compreensibilidade de um sistema, em conjunto com a redução do seu tempo de desenvolvimento.”

(Parnas, David L. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM, Vol. 15, No. 12, pp. 1053 – 1058, 1972. Tradução livre do autor.)

PROGRAMAÇÃO MODULAR

11

- Técnica de projeto (*design*) de software.
- Decomposibilidade de software:
 - Divisão do sistema em módulos.
- Cada módulo é uma unidade independente que provê uma funcionalidade específica de um sistema.
 - Módulos se comunicam.

PROGRAMAÇÃO MODULAR E POO

12

- Cada módulo é uma unidade independente que provê uma funcionalidade específica de um sistema.
- Módulos se comunicam.
 - Classe: é uma unidade independente?
 - Classe: tem uma funcionalidade específica?
 - Classe: se comunica com outras?

13

ESTAMOS AQUI PARA ...

RESOLVER PROBLEMAS



UM PROBLEMA A RESOLVER

14

- Iremos desenvolver um sistema simples de vendas e faturamento para um pequeno comércio.
- Como primeiro passo, precisamos representar os produtos que serão vendidos.
- Um produto tem uma descrição e um preço de custo. Seu preço de venda será dado pela adição de um valor ao preço de custo baseado em uma “margem de lucro” que vai de 10 a 50%.

UM PROBLEMA A RESOLVER

15

- Você entendeu as regras que determinam o funcionamento de um produto?
 - Métodos / ações.
- Você sabe quais dados o produto precisa ter?
 - Atributos / dados.
- Se você criar um produto, sabe quais respostas ele deve te dar?
 - Teste.

ORIENTAÇÃO POR OBJETOS

16

- Análise Orientada para Objetos (OOA/AOO):
 - Examina os requisitos de um sistema de uma perspectiva de classes e objetos, usando o vocabulário do domínio do problema.

ORIENTAÇÃO POR OBJETOS

17

- ▣ Projeto Orientado por Objetos (OOD/D00):
 - ▣ Projeto no qual o processo da decomposibilidade em objetos é utilizado e modelos físicos e lógicos de objetos são descritos.

ORIENTAÇÃO POR OBJETOS

18

- ▣ Programação Orientada por Objetos (OOP/P OO):
 - ▣ Implementação de programas organizados como coleções de objetos cooperativos.

ORIENTAÇÃO POR OBJETOS

19

- Análise orientada por objetos



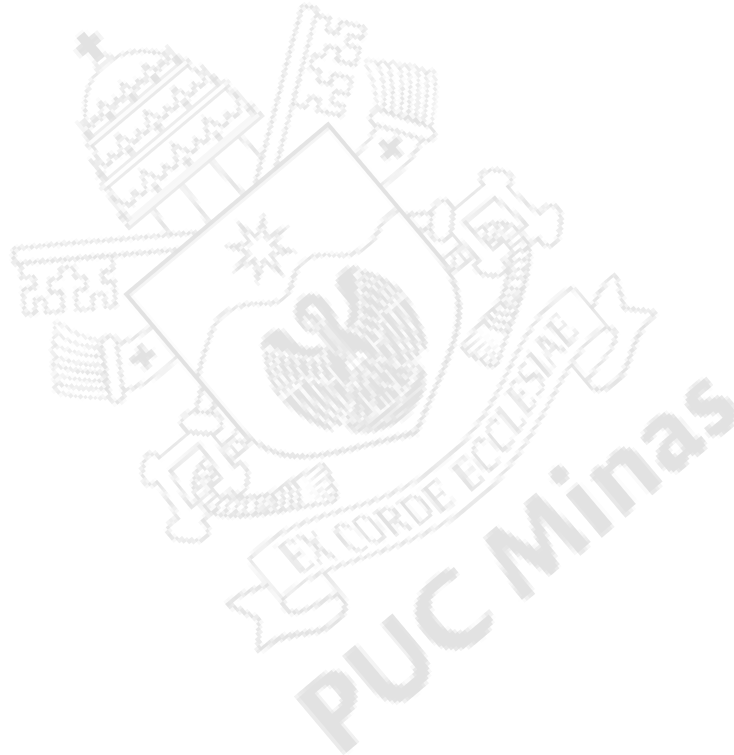
- Projeto orientado por objetos



- Programação orientada por objetos

CLASSE PRODUTO

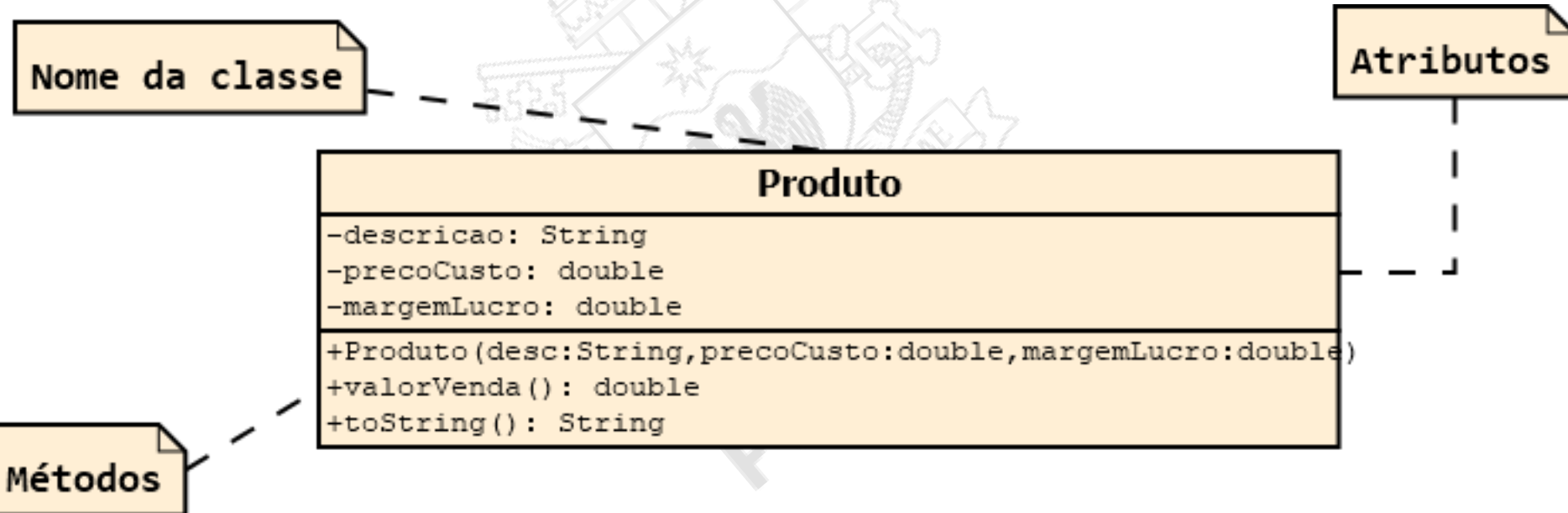
20



CLASSES: REPRESENTAÇÃO UML

21

▣ Classe Produto.



The background of the slide features a large, faint watermark of the PUC Minas logo. The logo is a crest with a star, a banner that reads "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

CRIANDO SOFTWARE...

23

- ▣ O que é um bom software para você?



CRIANDO SOFTWARE...

24

- O que é um bom software para você?
- Quando você cria um código, como você o avalia?

ENGENHARIA DE SOFTWARE

25

- ▣ Objetivo da Engenharia de Software:
 - ▣ Produção sistemática de software de boa qualidade.



- ▣ Produção sistemática:
 - ▣ Processo de desenvolvimento de software:
 - Especificação;
 - Desenvolvimento;
 - Teste/manutenção;
 - Evolução

ENGENHARIA DE SOFTWARE

27

■ Boa qualidade:

- Fatores externos: aqueles percebidos pelos usuários ou com os quais os usuários interagem.
- Fatores internos: aqueles relacionados com o projeto, com os quais os programadores interagem.

FATORES EXTERNOS

28

- 
- Correção
 - Robustez
 - Extensibilidade
 - Reusabilidade
 - Compatibilidade
 - Portabilidade
 - Eficiência
 - Facilidade de uso
 - Integridade
 - Verificabilidade

FATORES INTERNOS

29

- Fatores externos são os percebidos pelos usuários.
 - *Muito importante!!*
- Fatores externos são atingidos por meio dos fatores internos:
 - Legibilidade (+ Inteligibilidade);
 - Manutenibilidade;
 - Modularidade.

FATORES INTERNOS

30

- Fatores externos são os percebidos pelos usuários.
 - *Muito importante!!*
- Fatores externos são atingidos por meio dos fatores internos:
 - Legibilidade (+ Inteligibilidade);
 - Manutenibilidade;
 - Modularidade.

PROGRAMAÇÃO ORIENTADA POR OBJETOS

31

“I’m sorry that I long ago coined the term ‘objects’ for this topic because it gets many people to focus on the lesser idea. The big idea is ‘messaging’.”
(Alan Kay¹)

1 – Pai dos conceitos de POO, GUI, notebooks e da linguagem Smalltalk. Guitarrista e compositor profissional de jazz, cenógrafo profissional e organista clássico amador

PROGRAMAÇÃO ORIENTADA POR OBJETOS

32

- P00, segundo Alan Kay:
 - troca de mensagens;
 - proteção e retenção locais possibilitando ocultar o estado ou o processo;
 - associação tardia e dinâmica de tudo o que for possível.

PROGRAMAÇÃO ORIENTADA POR OBJETOS

33

- Paradigma orientado por objetos: abstrai o mundo real utilizando objetos que interagem entre si.
- Mecanismo para implementar a modularidade por meio do princípio da decomposibilidade:
 - Quais são os componentes de uma gestão de estoque?
 - Quais são os componentes de uma reunião virtual?

PROGRAMAÇÃO ORIENTADA POR OBJETOS

34

- Porém, como vimos, de nada criar objetos “de qualquer jeito” e sem pensar na dinâmica do sistema de software como um todo.

PRINCÍPIOS DE P00

35

- Pilares do paradigma de programação, utilizando seus conceitos fundamentais, para que um projeto atinja as metas que nortearam seu desenvolvimento.
- Metas fundamentais:
 - Redução de custo de desenvolvimento;
 - Aumento da qualidade de software.

SOLID PRINCIPLES¹

36

- S** *Single responsibility*
- O** *Open/Closed*
- L** *Liskov substitution*
- I** *Interface segregation*
- D** *Dependency inversion*

¹ - FEATHERS, Michael, a partir de:

MARTIN, Robert C. **Agile software development: principles, patterns, and practices**. Upper Saddle River, N.J.: Prentice Hall, c2003

MARTIN, Robert C; MARTIN, Micah. **Princípios, padrões e práticas ágeis em C#**. Porto Alegre: Bookman, 2011.

SINGLE RESPONSIBILITY - SRP

37

“Uma classe deve ter apenas um motivo para mudar.”

- Uma classe deve ter uma única responsabilidade.
 - Responsabilidade: motivo para mudança.

SINGLE RESPONSIBILITY - SRP

38

“Uma classe deve ter apenas um motivo para mudar.”

“Tente descrever uma classe com 25 palavras ou menos, sem usar “e” ou “ou”. Se você não consegue fazer isso, pode ser que você tenha mais de uma classe.”

WHITNEY, R. *Advanced Object-Oriented Design & Programming*. 2016

UM RECADO DO FOWLER

39

“Any fool can write code that a computer can understand... But only good programmers write code that humans can understand.”

(Martin Fowler)



40

BÔNUS!!

CUIDADO AO PROGRAMAR

QUALIDADE DE SOFTWARE

41

- Programando descuidadamente:
 - Ignorar as boas práticas;
 - Codificar antes de pensar;
 - Nomenclatura mal feita;
 - Código duplicado;
 - Programar sem testar;
 - Reaproveitar sem entender;

QUALIDADE DE SOFTWARE

42

- Programando descuidadamente:
 - Não usar ou usar incorretamente o controle de versões;
 - Ignorar avisos e erros;
 - Não tratar entrada de dados;
 - Pouca documentação;
 - (Muita) documentação substituindo refatoração;
 - Alto acoplamento/dependência (projeto espaguete).

CÓDIGO DE QUALIDADE

43

- ▣ Don't repeat yourself (DRY);
 - ▣ *Não se repita*
- ▣ Comment where needed;
 - ▣ *Comentários somente onde necessário*
- ▣ Fail fast (code should reveal its bugs as early as possible);
 - ▣ *Falhe rapidamente (descubra os bugs o quanto antes)*

Miller, Rob. Software Construction: Code Review (curso online).
Disponível em <https://web.mit.edu/6.102/www/sp23/classes/03-code-review/>
Acesso em 06/08/2024

CÓDIGO DE QUALIDADE

44

- ▣ Avoid magic numbers;
 - ▣ *Evite números mágicos*
- ▣ One purpose for each variable;
 - ▣ *Um propósito para cada variável*
- ▣ Use good names;
 - ▣ *Use bons nomes*

Miller, Rob. Software Construction: Code Review (curso online).
Disponível em <https://web.mit.edu/6.102/www/sp23/classes/03-code-review/>
Acesso em 06/08/2024

CÓDIGO DE QUALIDADE

45

- Use whitespace and punctuation to help the reader;
 - *Use espaços e pontuação para facilitar ao leitor*
- Don't use global variables;
 - *Não use variáveis globais*
- Functions should return results, not print them;
 - *Métodos devem retornar resultados, não imprimí-los*
- Avoid special-case code.
 - *Evite códigos para casos especiais*

Miller, Rob. Software Construction: Code Review (curso online).
Disponível em <https://web.mit.edu/6.102/www/sp23/classes/03-code-review/>
Acesso em 06/08/2024

OBRIGADO.

DÚVIDAS?

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE